

Práctica 2.2: Explotación Paralelismo de Hilos

Índice

Objetivos	1
Ejercicio 1: Hello World	1
Ejercicio 2: Directivas de trabajo compartido	2
Expresión de paralelismo	3
ToDo	3
Ejercicio 3: Condiciones de Carrera	3
Ejercicio 4: Tareas OMP	5
A tener en cuenta	5
Tareas a realizar por el alumno	6
Tareas a considerar	6
Ejercicio 5: Quick-sort	6
Ejercicio 6: K-means	6
¿Como funciona el algoritmo?	6
ToDo	8
Herramienta de perfilado VTune	9
Intel VTune	9
0.0.1 Comienzo Herramienta	9
Opciones de análisis	9
Intel VTune: Análisis Hotspots	10
Intel VTune: Parallelism Analysis	10
Intel VTune: Threading Efficiency	11
Intel VTune: Threading Efficiency	11
Intel VTune: Microarchitecture	13

Objetivos

Esta práctica persigue que los estudiantes se familiaricen con la mejora de rendimiento por la explotación de paralelismo TLP mediante el uso de OpenMP. Este esquema de paralelismo es compatible con los estudiados hasta el momento en la asignatura.

Ejercicio 1: Hello World

El código “hello.c” versará sobre la compilación y control de hilos de ejecución

```
#include <stdio.h>
#include <omp.h>

int main() {
```

```
// This code is executed by 1 thread
printf("OpenMP with %d max threads\n", omp_get_max_threads ());
printf("OpenMP with %d procs available\n", omp_get_num_procs ());

#pragma omp parallel
{
// This code is executed in parallel
// by multiple threads
printf("Hello World from thread %d of %d threads\n",
omp_get_thread_num(), omp_get_num_threads ());
}
}
```

- Pudiendo compilar con ambos compiladores y generando el fichero de reportes

```
user@lab:~ $ icx -o hello.icx hello.c -qopenmp -qopt-report=1
user@lab:~ $ more hello.optrpt
```

```
...
COMPILE FUNC: main
-> BROKER: __kmpc_fork_call hello.c(14,2) (main.extracted)
```

- Para controlar el número de hilos de ejecución se puede hacer de varias formas:
 - Con la variable de entorno `OMP_NUM_THREADS`
 - En la pragma `#pragma omp parallel num_threads(3)`

```
user@lab:~ $ export OMP_NUM_THREADS=3
user@lab:~ $ ./hello.icx
OpenMP with 3 max threads
OpenMP with 4 procs available
Hello World from thread 0 of 3 threads
Hello World from thread 1 of 3 threads
Hello World from thread 2 of 3 threads
```

- Para controlar el tipo de cores (P-cores o E-cores) donde ejecutar el código se recomienda emplear el comando `taskset`

```
user@lab:~ $ OMP_NUM_THREADS=2 taskset --cpu-list 0,1 ./hello.icx
OpenMP with 2 max threads
OpenMP with 2 procs available
Hello World from thread 0 of 2 threads
Hello World from thread 1 of 2 threads
```

Ejercicio 2: Directivas de trabajo compartido

- En OpenMP existen varias formas de explicitar la distribución del trabajo entre los hilos
 - `#pragma omp for/#pragma omp parallel for` para distribuir las iteraciones de los bucles entre los hilos disponibles
 - `#pragma omp sections` para crear secciones de código concurrente entre hilos
 - `#pragma omp single` para expresar una región secuencial dentro de una región paralela
- En este ejemplo, el código **prime.c** trabaja dos aspectos:
 - Distribución de trabajo entre hilos
 - Gestión de variables (privadas, compartidas...)
- El ejemplo “prime.c” calcula los números primos menores a un determinado **n** (parámetro de entrada)
 - **i** es primo si no tiene divisores ($2, i/2$)
 - `#define DEBUG` visualiza la lista de primos

```
//#pragma omp parallel...
for (i=2; i<n; i++)
{
    not_flag = 0;
    j=2;
    while(j<=i/2 && !not_flag)
    {
        if(i%j==0) // not prime
            not_flag=1;
        j++;
    }
    if (j>=i/2 && !not_flag)
        primes[++k] = i;
}
```

Expresión de paralelismo

- Iteraciones del bucle **i** potencialmente paralelas
 - Iteraciones independientes
- A tener en cuenta
 - Uso de variables: privadas, compartidas... ect
 - Variable ++k es el índice de la lista de números primos
 - * Posible carrera: problema **read&update&write**
 - * Solución: `#omp critical`
 - * Lista desordenada: implementar un `sort(primes)`

ToDo

- **A Evaluar** Se recomienda estudiar el impacto de las diferentes políticas de distribución de carga
 - *STATIC*
 - *STATIC, chunk*
 - *DYNAMIC[, chunk]*
 - *GUIDED[, chunk]*
 - *AUTO*

Ejercicio 3: Condiciones de Carrera

- El tercer ejercicio se basa en el código “trapezoidal.c” que calcula la integral mediante el método del trapecio de acuerdo a la siguiente ecuación
 - Area del trapecio $S = \frac{1}{2}(f(a') + f(b')) * h$
 - Integral como suma de trapecios: $\int_a^b f(x) \partial x = \sum \frac{f(a') + f(b')}{2} h$
- Podemos destacar **dos tipos tareas**:
 - Cálculo de areas de cada trapecio individual
 - Acumulación de trapecios (*integral*) donde potencialmente pueden aparecer condiciones de carrera

```
double Trap(double a, double b, int n, double h) {
    double integral, area;
    int k;

    integral = 0.0;
    for (k = 1; k <= n; k++) {
        area = h * (f(a+k*h) + f(a+(k-1)*h)) / 2.0;
        integral += area;
    }
}
```

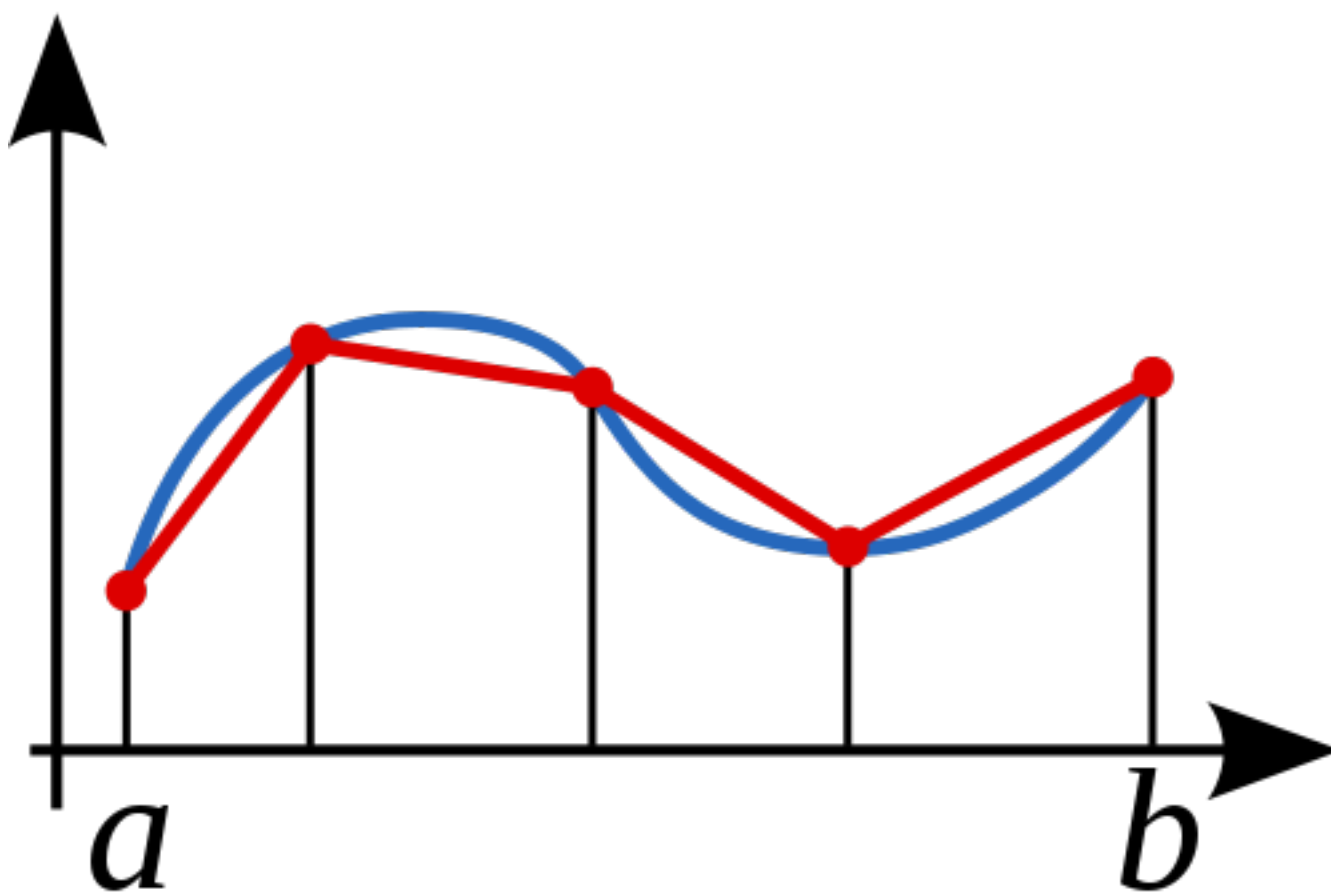


Figure 1: Aproximación de integral a trapezoide

```

    return integral;
} /* Trap */

```

Ejercicio 4: Tareas OMP

- El cuarto ejercicio se basa en “fibonacci.c” donde se pone en valor la explotación del paralelismo mediante tareas haciendo uso del algoritmo de Fibonacci
 - Tareas OMP = unidades de trabajo (ejecución puede diferirse)
 - Las tareas se componen de:
 - * código para ejecutar y datos
 - Hilos pueden **cooperar** para ejecutarlas

```

long comp_fib_numbers(int n)
{
    long fnm1, fnm2, fn;
    if ( n == 0 || n == 1 ) return(1);
    if ( n<20 ) return(comp_fib_numbers(n-1) +comp_fib_numbers(n-2));

    #pragma omp task...
    {fnm1 = comp_fib_numbers(n-1);}
    #pragma omp task...
    {fnm2 = comp_fib_numbers(n-2);}
    #pragma omp...
    fn = fnm1 + fnm2;

    return(fn);
}

int main(int argc, char* argv[]) {
    ....
    #pragma omp parallel
    {
        #pragma omp single
        fibo = comp_fib_numbers(n);
    }
    ...
} /* main */

```

A tener en cuenta

- Posible carrera: problema **read&update&write** en la variable *integral*
 - Estudiar las diferentes alternativas:
 - * Uso de `atomic`, `critical`
 - * Expresión de paralelismo con `#pragma omp for` pero usando la cláusula **reduction**

```

#pragma omp parallel private(...) firstprivate(...) shared(integral)
{
    id = ...
    nth = ...
    n_per_thread = n/nths;

    k_init_thread = ...
    k_end_thread = ...

    integral = 0.0;
    for (k = k_init_thread; k <= k_end_thread; k++) {
        area = h*(f(a+k*h)+f(a+(k-1)*h))/2.0;
    }
}

```

```

        integral_thread += area;
    }

    #pragma omp critical
    {integral += integral_thread;}

}

return integral;

```

Tareas a realizar por el alumno

- Las tareas a entregar por el alumno corresponde a la implementación paralela mediante el paradigma de programación de OpenMP sobre de tres algoritmos:
 1. Quick-sort
 2. Kmeans

Tareas a considerar

- Paralelizar los códigos
 - Se recomienda utilizar la herramienta Intel vTune para encontrar los cuellos de botella y evaluar la escalabilidad de la paralelización OpenMP
 - No olvides que puedes combinar paralelismo del tipo SIMD con paralelismo multi-hilos en los cores (ambos soportados por OpenMP)

Ejercicio 5: Quick-sort

- Algoritmo de ordenación basado en el concepto divide y vencerás
 1. Elegir un elemento al que llamaremos pivote.
 2. Mover los demás elementos de la lista a cada lado del pivote
 - La lista inicial está separada en dos sublistas: con elementos menores y mayores al pivote
 3. Repetir el proceso de forma recursiva para cada sublista

El código de **quicksort.cpp** hace una llamada a la función `quickSort` que calcula la posición del pivote (invocación a `partition`) lo que permite realizar llamadas recursivas a `quickSort` para cada una de las sublistas:

```

void quickSort(int arr[], int low, int high){
    if(low < high){
        int pivot = arr[high];
        int pos = partition(arr, low, high, pivot);

        quickSort(arr, low, pos-1);
        quickSort(arr, pos+1, high);
    }
}

```

Ejercicio 6: K-means

El K-Means es un algoritmo de aprendizaje no supervisado utilizado para la agrupación de datos en K clusters. Su objetivo es minimizar la suma de las distancias cuadradas entre los puntos de datos y el centroide más cercano.

¿Como funciona el algoritmo?

1. *Inicialización*: Se eligen K centroides de manera aleatoria.

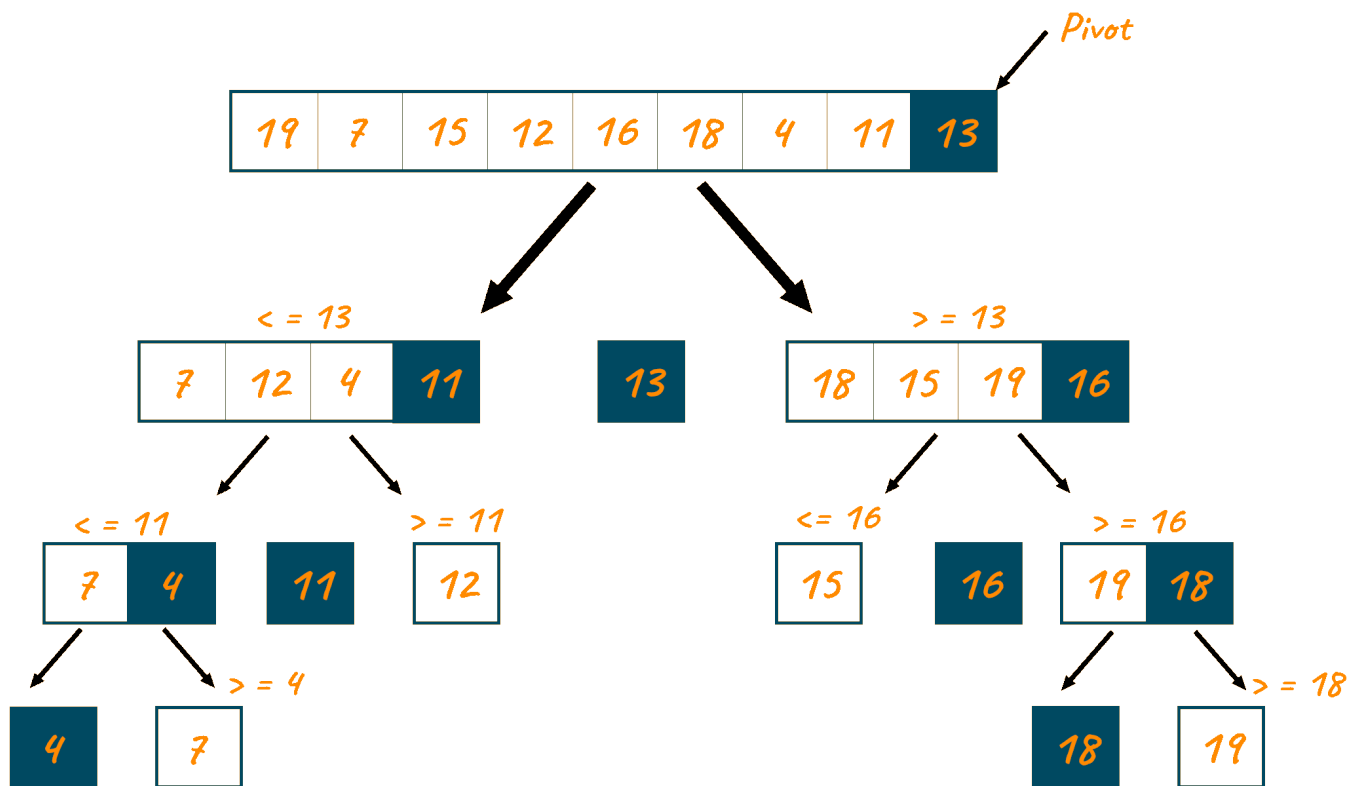


Figure 2: Algoritmo QuickSort

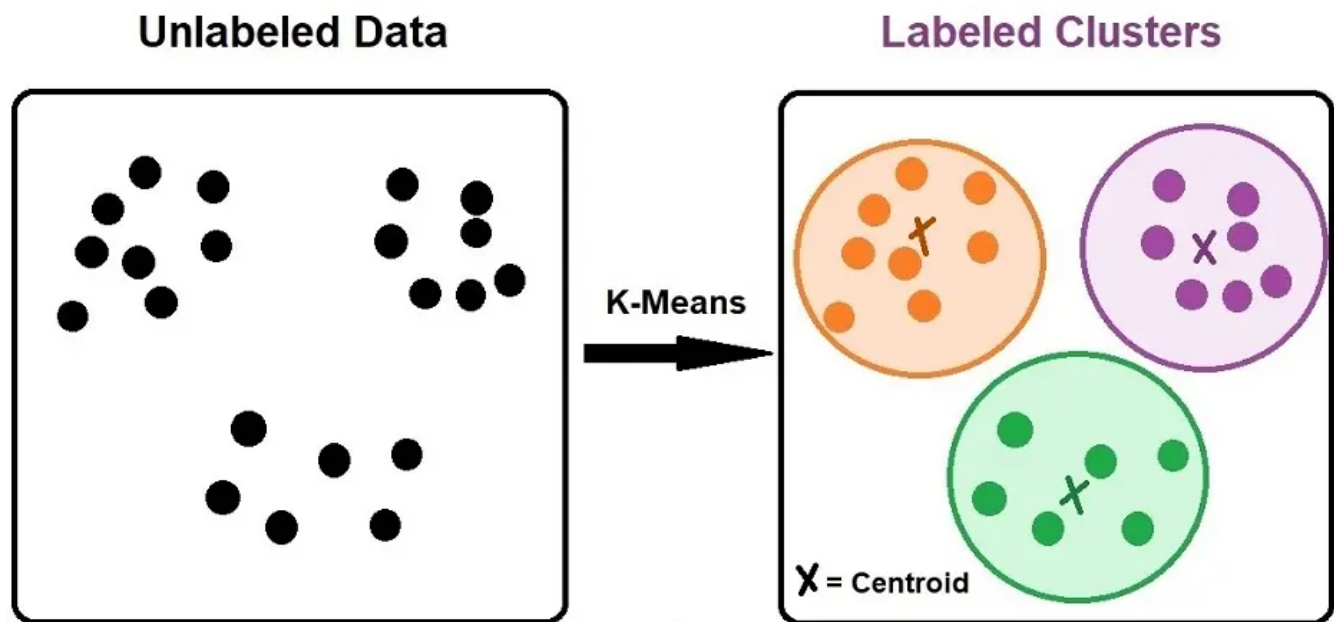


Figure 3: Algoritmo K-means

2. *Asignación del Clúster*: Cada punto se asigna al centroide más cercano.
3. *Reubicación de Centroides*: Se calculan los nuevos centroides como el promedio de los puntos asignados a cada cluster.
4. *Repetición*: Se repiten los pasos 2 y 3 hasta que los centroides no cambian significativamente (convergencia).

La implementación del algoritmo se realiza en el fichero **kmeans.cpp** donde la función `kmeans` es la encargada de realizar las etapas anteriormente mencionadas:

- La etapa de inicialización se realice en la función `initializeCentroid` donde se seleccionan como centroides algunos de los puntos del dataset almacenado en la estructura `data`
- La asignación de cluster se procesa en la función `assignCluster` donde se calcula las distancias euclídeas a los centroides y se queda con la distancia menor a todos los centroides
- La actualización de los centroides se realiza como la media de las posiciones asociadas a cada clase en la función `updateCentroid`

```
void kmeans(std::vector<Point>& data, int k, int max_iters = 20000) {
    int dimensions = data[0].coords.size();
    std::vector<Centroid> centroids(k);
    initializeCentroids(centroids, data, k);

    for (int iter = 0; iter < max_iters; iter++) {
        assignClusters(data, centroids, k);

        std::vector<Centroid> newCentroids = centroids;
        updateCentroids(data, newCentroids, k, dimensions);

        centroids = newCentroids;
    }

    std::cout << "Final cluster centers:\n";
    for (int i = 0; i < k; i++) {
        std::cout << "Cluster " << i << ": (";
        for (size_t d = 0; d < centroids[i].coords.size(); d++) {
            std::cout << centroids[i].coords[d] << (d == centroids[i].coords.size() - 1 ? ")" : ", ");
        }
        std::cout << "\n";
    }
}
```

El contenido tanto de los puntos del dataset a entrenar como el de los centroides es una estructura con las coordenadas de cada punto N-dimensional y el tipo de cluster que viene descrito en el fichero `*csv_dataset.h`:

```
struct Point {
    std::vector<double> coords;
    int cluster;
};

struct Centroid {
    std::vector<double> coords;
};
```

ToDo

- Realizar una implementación en OpenMP que explote el paralelismo TLP mediante hilos teniendo en cuenta:
 - La función `assignCluster` puede paralelizarse sencillamente ya que el cálculo de la distancia a los centroides y la selección del centroide más cercano es una tarea completamente independiente
 - La función `updateCentroids` puede también paralelizarse pero presenta un reto mayor: **existen varias reducciones** como el proceso en el array `count[cluster]`, o `sumCoords[cluster][d]`

Herramienta de perfilado VTune

Intel VTune

- Intel® VTune™ Profiler
- Como mejorar el rendimiento con varios análisis
 - Hotspots
 - Threading Efficiency
 - Microarchitecture
 - Memory Access
- Variables de entorno

```
user@lab:~ $ export ONEAPI_PATH=/opt/intel/oneapi/  
user@lab:~ $ source $ONEAPI_PATH/vtune/latest/env/vars.sh
```

- La Herramienta Intel Vtune Amplifier nos permite realizar un perfilado de la aplicación paralela para detectar posibles mejoras
 - Lanzamiento herramienta gráfica `amplxe-gui`, más moderno **vtune-gui**
 - Lanzamiento por línea de comandos `amplxe-cl`, más moderno **vtune-collect**

0.0.1 Comienzo Herramienta

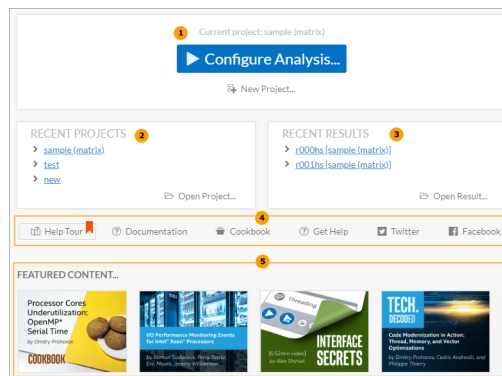


Figure 4: Inicio VTune

1. Proyecto actual con el análisis de configuración (*New Project*)
2. Proyectos recientes
3. Resultados recientes
4. Recursos y documentación online
5. Artículos de y últimas noticias

Opciones de análisis

- Hotspots
 - Utilización de CPU
- Memory Consumption:
 - Consumo de memoria (malloc-free) en cada rutina
- Microarchitecture Exploration:
 - Analiza cuellos de botella que afectan al rendimiento
 - Contadores HW
- Memory Access
 - Identifica los accesos a jerarquía memoria (NUMA, DRAM, L2...)

- Análisis
 - Más info en web de Intel-VTune y vídeo
 - A la hora de crear el proyecto con *New Project* seleccionar *Local Host* para analizar en el equipo local
 - * Introducir el ejecutable (*path*) y primer análisis *Hotspots*

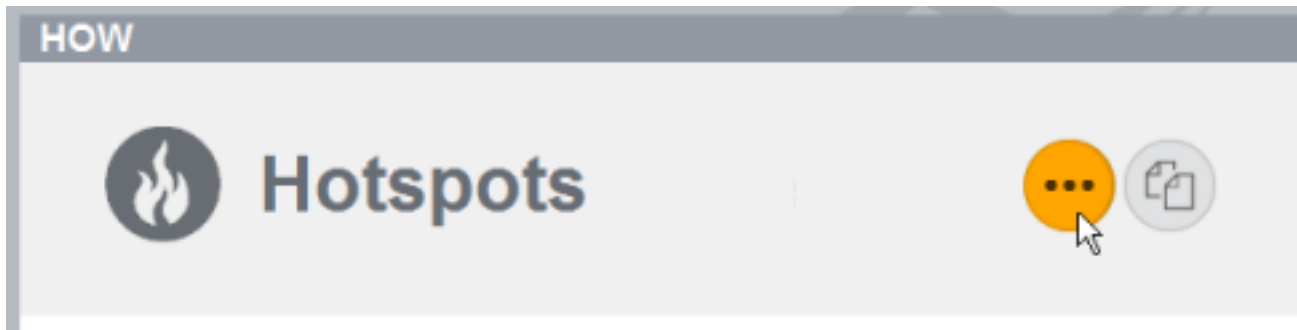


Figure 5: Análisis VTune

Intel VTune: Análisis Hotspots

- *Hotspots* dirigido a la optimización de software y conocer dónde pasa tiempo la aplicación: **analizar la eficiencia**
- Incluye los análisis:
 - *Hotspots* para identificar las funciones más costosas y muestra la actividad de cada hilo
 - *Memory consumption* para analizar el consumo de memoria: caches y RAM

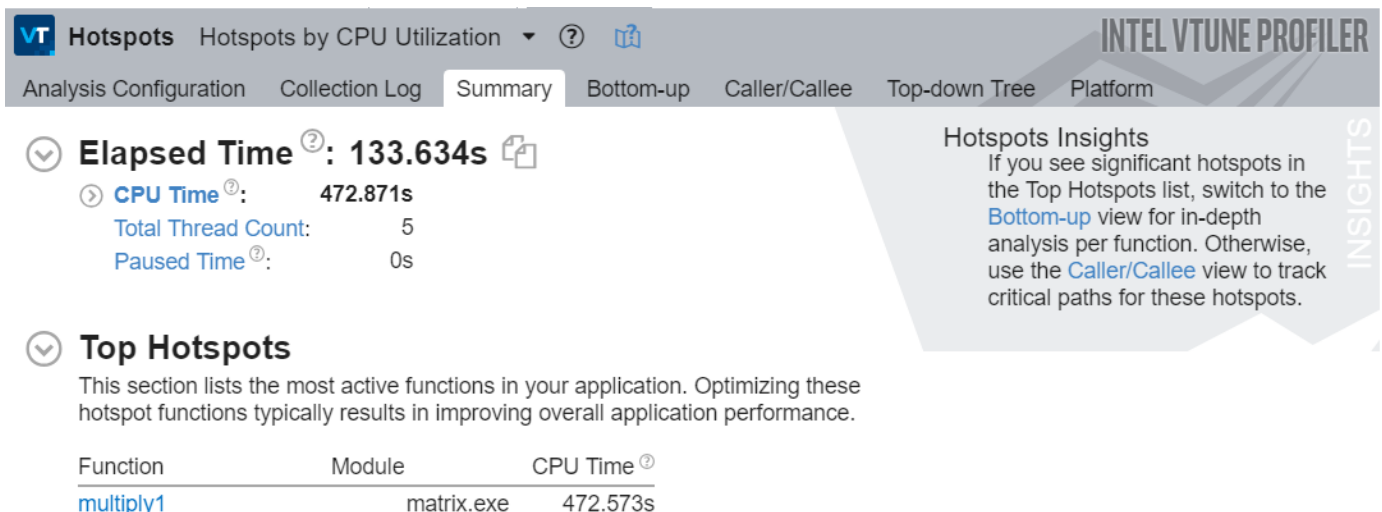


Figure 6: Análisis Hotspot

Intel VTune: Parallelism Analysis

- Tipos de análisis para aplicaciones sensibles en cómputo: análisis general del rendimiento
- Incluye los análisis:
 - *Threading*: muestra balanceo de trabajo entre hilos y puntos de sincronización
 - *Compute-intensive Application*: caracterización rendimiento de HPC (punto flotante y la eficiencia de la memoria)

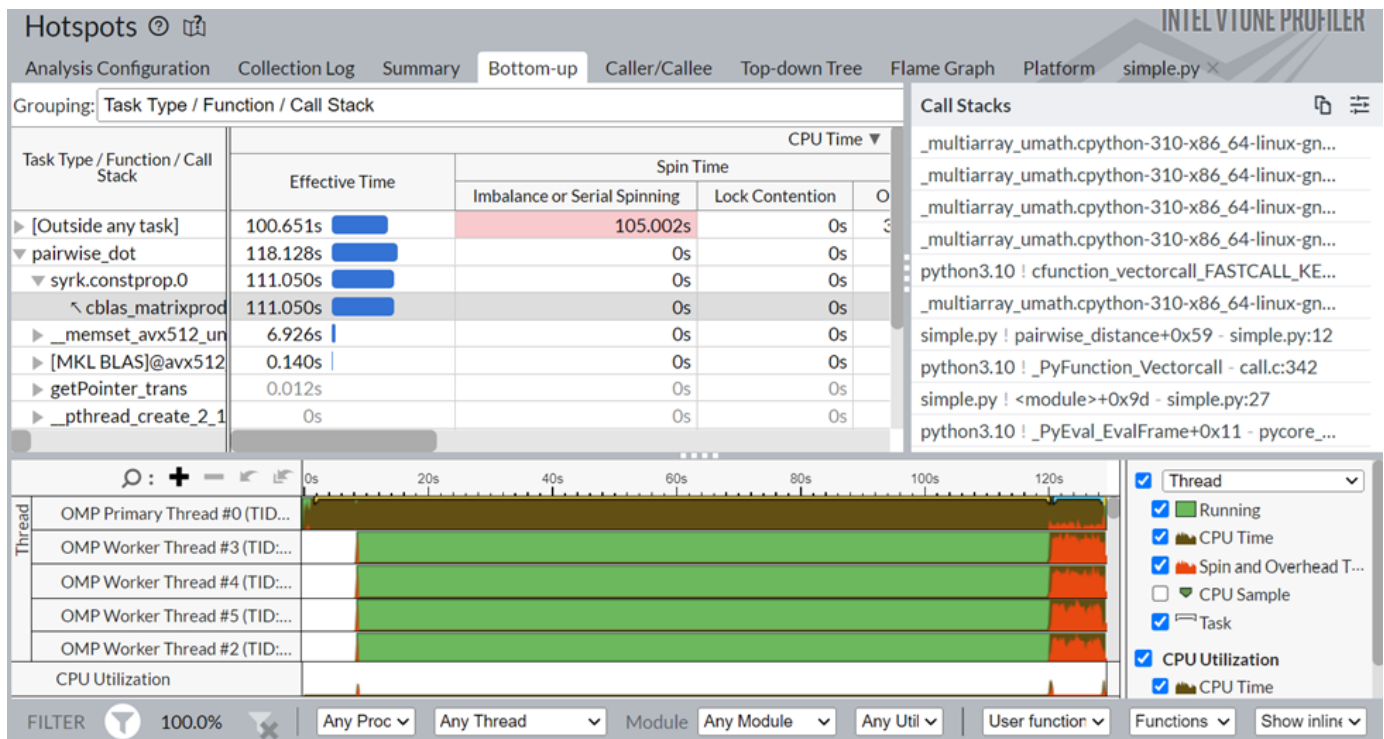


Figure 7: Análisis Paralelismo

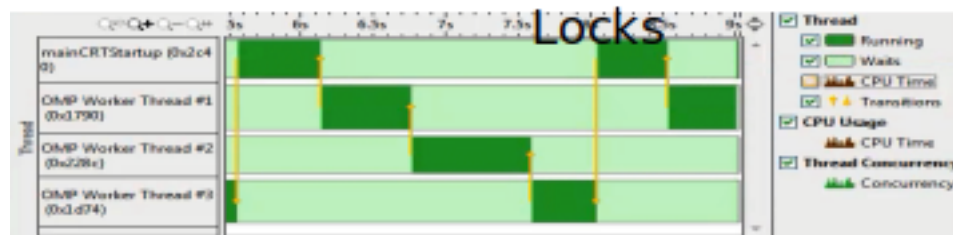
Intel VTune: Threading Efficiency

- Permite explorar
 - Wait Time: esperas prologadas por regiones de sincronización
 - Spin y Overhead Time: esperas y sobrecoste asociado al manejo de las regiones paralelas
 - Thread count: tiempo espera debido a sobresubscripción, espera a que los recursos compartidos estén disponibles cuando se ejecutan más hilos lógicos que cores físicos
- Problemas asociados al paralelismo
 - Asegurarse que se ejecutan todos los hilos disponibles
 - Problemas comunes a la concurrencia pueden identificarse
 - Análisis para detectar la contención en operaciones tipo Locks/Waits

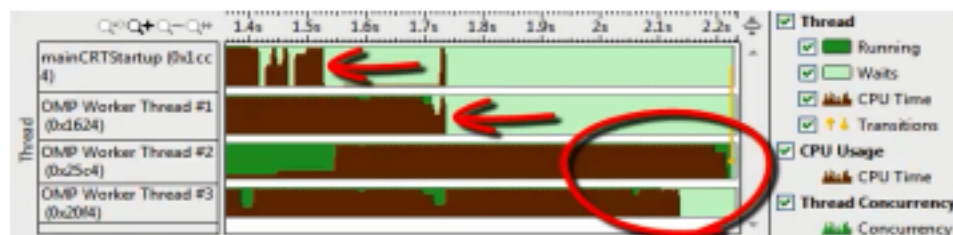
Intel VTune: Threading Efficiency

- Demo:
 - Paralelización bucle for

Coarse-Grain



Thread Imbalance



High Lock Contention

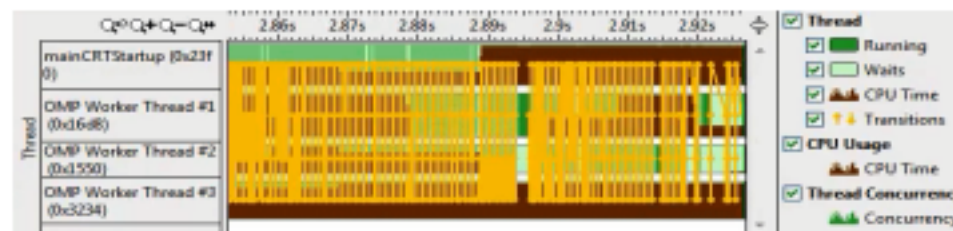
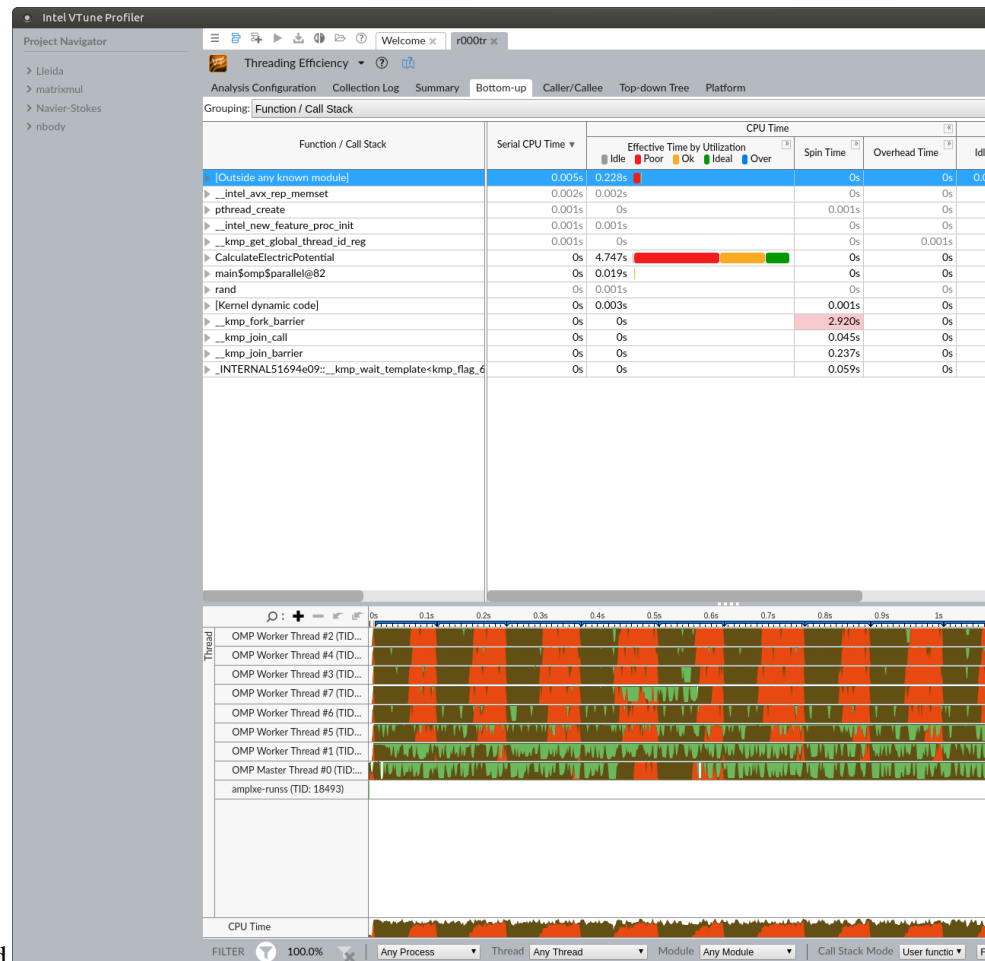


Figure 8: Eficiencia de Hilos

- schedules: static, dynamic, guided



Intel VTune: Microarchitecture

- **Microarchitecture analysis Group** introduce un tipo de análisis que ayuda a estimar los motivos de las ineficiencias en los procesadores modernos
 - *Microarchitecture Exploration* ayuda a identificar los problemas más comunes que afectan al rendimiento de la aplicación. Se puede considerar este análisis como punto de partida al análisis a nivel hw
 - *Memory Access* mide una serie de métricas para identificar accesos a los diferentes niveles de la jerarquía de memoria (como por ejemplo en arquitecturas NUMA)
- Una vez completado el análisis **Hotspots** para conocer ineficiencias en tu código...
 - Se recomienda efectuar el análisis *Microarchitecture Exploration analysis* para comprender como las ineficiencias se manifiestan en el uso del pipeline del core
 - VTune Profiler recolecta una lista complete de eventos hw
 - Calcula unas métricas predefinidas = identifica a nivel hw problemas asociados a ineficiencias
- Problemas relacionados:
 - Backend bound
 - * Uops posibles que no se puede lanzar al Fetch+Dec
 - Retiring
 - * Uops completadas (1uop/cycle)
 - Bad speculation
 - * Uops especuladas de forma incorrecta y no “retiradas”
 - Backend bound
 - * Uops completadas, pero no 1 por ciclo (fallos cache)

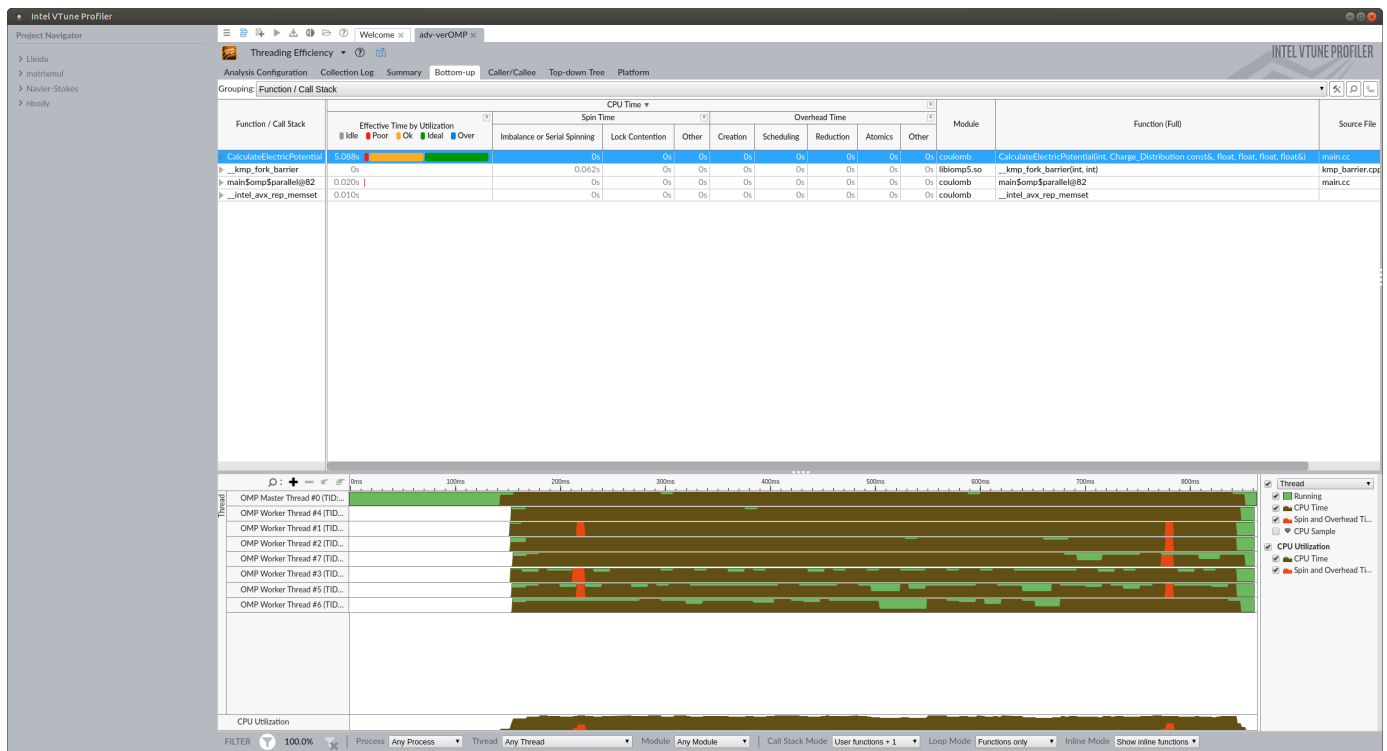


Figure 10: Ejemplo Nbody-coulomb: guided

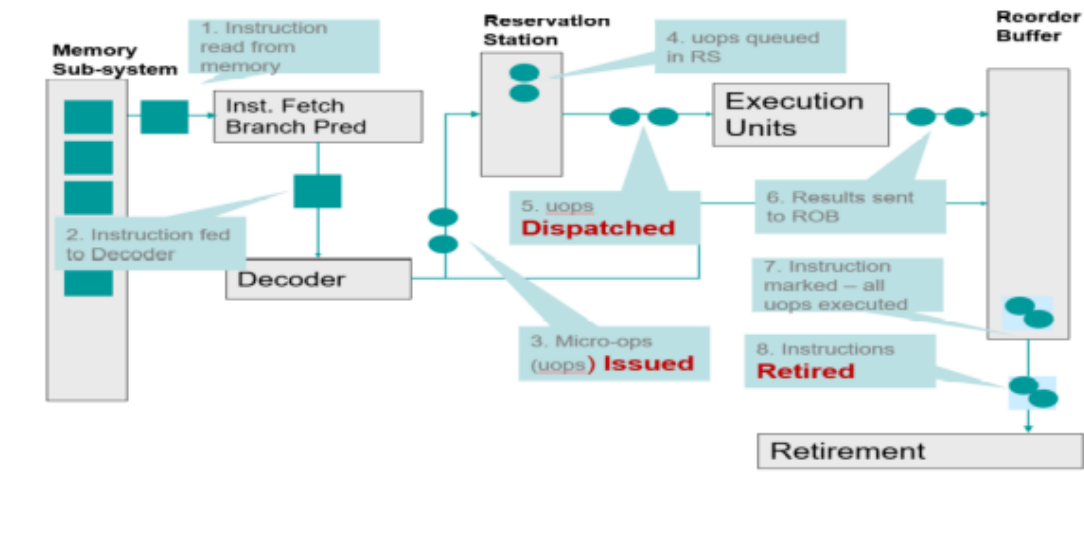


Figure 11: Flujo de Instrucciones

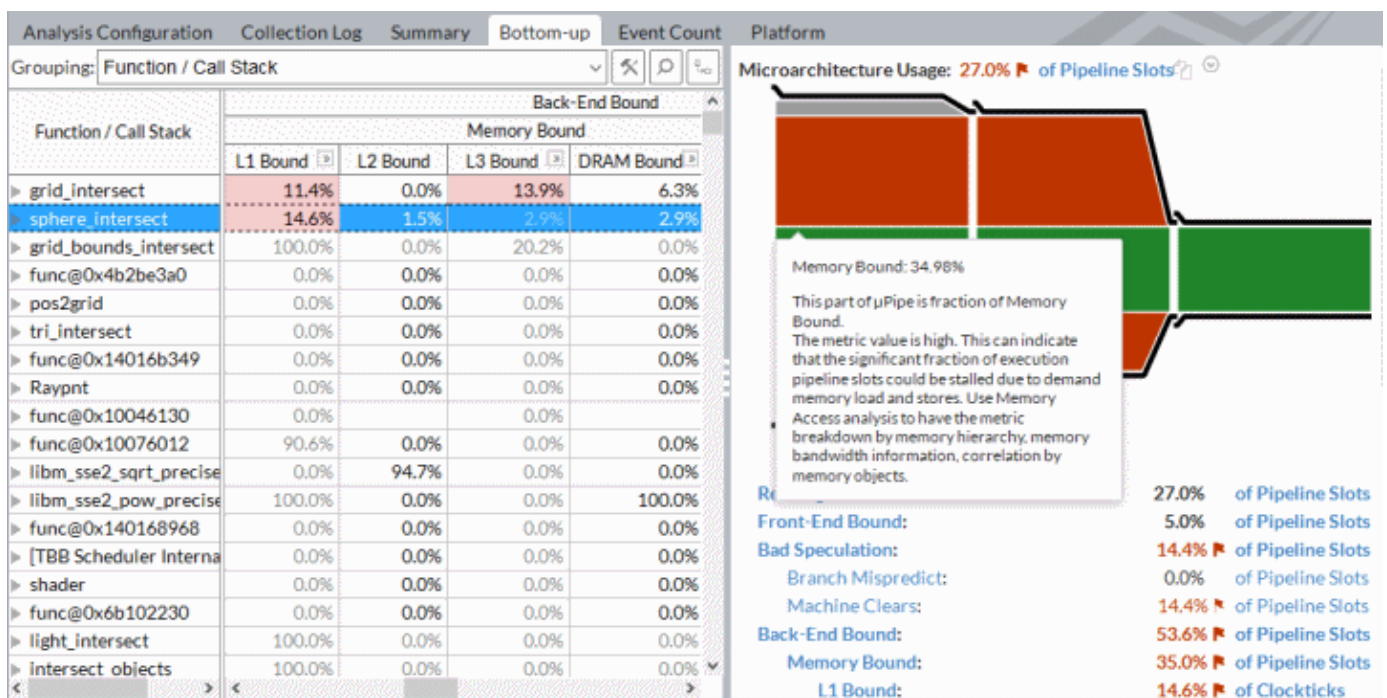


Figure 12: Análisis Microarquitectura